

PixelDex

Integrantes:

- Konrado Ribeiro - RA: 10435499
- Leonardo Valencio Dourado - RA: 10736460
- Luiz Filipe Batista dos Santos - RA: 10438938

Prof: Traue

Disciplina: Estrutura de Dados

Projeto N2 - Relatorio

Arquitetura do Sistema

O sistema foi desenvolvido seguindo uma arquitetura modular, onde cada parte possui uma responsabilidade clara. As principais estruturas são:

a) Estrutura de Telas (Screens)

O projeto utiliza o padrão de telas para organizar o fluxo do jogo. Cada estado do jogo é representado por uma classe específica, como:

- StartScreen – tela inicial do jogo.
- GameScreen – tela principal de execução do jogo.
- Outras telas auxiliares (caso existam) para menus, configurações etc.
- Isso facilita a manutenção, evita código duplicado e permite trocar de telas de forma simples.

b) Classe Principal (Main / Game)

A classe principal inicializa o jogo e define qual tela será carregada primeiro. Ela funciona como um controlador geral, carregando os recursos essenciais e repassando o controle para as telas.

c) Organização em Módulos

Os componentes principais do sistema são separados em:

- Lógica do jogo: movimentação, regras, ações.
- Renderização: desenho dos sprites, animações e efeitos visuais.
- Entrada do usuário: leitura de teclado, mouse ou toque.
- Recursos (assets): imagens, sons, música, arquivos de configuração.
- Cada parte pode ser alterada sem mexer no resto, seguindo o conceito de baixo acoplamento.

Análise de Complexidade

a) Complexidade da Lógica Principal

A maior parte das ações internas do jogo ocorre dentro de loops de atualização (game loop).

Esses loops processam:

- colisões
- movimentação
- atualização de estados
- renderização dos sprites

Normalmente, cada ciclo executa operações $O(n)$, onde n = quantidade de objetos na cena.

Isso significa que, quanto mais elementos existirem na tela ao mesmo tempo, maior será o custo por frame.

b) Ocupação de Memória

O uso de imagens, animações e sons afeta diretamente a memória.

Sprites grandes ou muitos assets carregados ao mesmo tempo aumentam o consumo, podendo reduzir a performance em máquinas mais fracas.

c) Estruturas Internas

As estruturas de controle (listas, arrays, loops) possuem custo linear na maior parte das operações.

Não há algoritmos complexos, como buscas profundas ou grafos, então a complexidade geral do projeto se mantém entre:

- $O(n)$ para atualizações
- $O(1)$ para ações simples (movimento, input)

Limitações do Sistema

a) Escalabilidade

-O desempenho pode diminuir caso:

- muitos sprites sejam renderizados simultaneamente;
- haja animações de alta resolução;
- o número de colisões simultâneas aumente.
- Como o loop roda constantemente, qualquer aumento no número de objetos impacta diretamente o FPS.

b) Dependência de Assets

Imagens pesadas, sons ou músicas grandes podem:

- aumentar tempo de carregamento;
- ocupar mais memória;
- causar travamentos em dispositivos mais fracos.

c) Limitações de Plataforma

A performance e comportamentos gráficos podem variar conforme:

- navegador
- sistema operacional
- hardware
- limitações do framework utilizado (LibGDX, Phaser, etc.)

d) Falta de Modularidade Avançada

Embora organizado, o projeto pode se tornar mais robusto caso:

- os elementos do jogo sejam convertidos em entidades separadas;
- seja aplicado um padrão como ECS (Entity Component System);
- haja centralização de eventos e colisões para facilitar expansão.